



Từ Port Scanning đến thiết lập Reverse Shell

Network Thực Chiến · Series: Debug Mạng từ A–Z · Tập 04

Nội dung

1. **nc là gì?** — Raw TCP/UDP pipe, 2 phiên bản cần biết
2. **Test port** — Thay thế ping khi ICMP bị chặn
3. **Server tạm thời** — Dựng listener không cần cài app
4. **Transfer file** — Không cần SSH/SCP
5. **HTTP thủ công & Banner grabbing** — Hiểu protocol ở tầng thô
6. **Relay / Pipe** — Debug firewall rule phức tạp
7. **Kịch bản thực chiến** — Firewall, K8s NetworkPolicy, UDP
8. **Giới hạn** — Khi nào dùng công cụ khác

Phần 1

nc là gì?

nc — Pipe thẳng vào socket

Hầu hết debug tool có protocol riêng: `curl` nói HTTP, `ssh` nói SSH, `mysql` nói MySQL.

`nc` không có protocol nào cả.

```
nc = stdin/stdout ↔ TCP/UDP socket
```

```
Gửi gì → gửi thẳng ra socket (raw bytes)
```

```
Nhận gì từ socket → in ra stdout (raw bytes)
```

Chính vì vậy `nc` dùng được để:

- Test bất kỳ port nào (TCP hoặc UDP)
- Giả lập server hoặc client của bất kỳ protocol nào
- Transfer dữ liệu thô không cần SSH
- Debug firewall rule mà không cần deploy app thật

Hai phiên bản — Cú pháp khác nhau

	netcat-openbsd	ncat (nmap)
Lệnh	nc	ncat
Ubuntu 24.04	✓ Mặc định	✗ Cài thêm: apt install ncat
Listen	nc -l 9999	ncat -l 9999
Keep-open	✗	✓ ncat --keep-open
TLS	✗	✓ ncat --ssl

```
# Kiểm tra đang dùng phiên bản nào
nc -h 2>&1 | head -1
# "OpenBSD netcat (Debian patchlevel 1.219-1ubuntu2)" → netcat-openbsd ← Ubuntu 24.04 default
# "Ncat: Version 7.94 ( https://nmap.org/ncat )" → ncat (nmap)
```

Bài này demo trên Ubuntu 24.04 với netcat-openbsd. Lưu ý điểm khác biệt khi dùng ncat.

Phần 2

Test Port

Test port — Thay thế ping khi ICMP bị chặn

`ping` dùng ICMP. Firewall doanh nghiệp và cloud thường chặn ICMP — `ping` fail không có nghĩa là host chết.

```
# Kiểm tra TCP port (-z: zero I/O, không gửi data; -v: verbose)
nc -zv 192.168.1.1 22

# Kết quả thành công:
# Connection to 192.168.1.1 22 port [tcp/ssh] succeeded! ✓

# Kết quả thất bại:
# nc: connect to 192.168.1.1 port 22 (tcp) failed: Connection refused ✗
# nc: connect to 192.168.1.1 port 22 (tcp) failed: No route to host ✗
```

Phân biệt 2 loại lỗi:

Lỗi	Nguyên nhân
Connection refused	Host tới được, nhưng không có gì listen trên port đó
No route to host / timeout	Firewall/Security Group đang chặn, hoặc host không tồn tại

💡 `nc` thay thế `telnet` cho port testing: `telnet host port` → `nc -zv host port`. Telnet không dùng trong production (plain-text, deprecated).

Test nhiều port — Port scanning nhẹ

```
# Scan dài port
nc -zv 192.168.1.1 20-25
# Connection to 192.168.1.1 22 port [tcp/ssh] succeeded!
# nc: connect to 192.168.1.1 port 20 (tcp) failed: Connection refused
# nc: connect to 192.168.1.1 port 21 (tcp) failed: Connection refused
# ...

# Timeout – tránh treo khi host không phản hồi
nc -zv -w 3 192.168.1.1 8080
# Fail sau 3 giây, không chờ mãi

# UDP port test
nc -zuv 192.168.1.1 53
# ⚠ UDP không có handshake → "succeeded" không đảm bảo port thực sự mở
# Cần app-level response để xác nhận (VD: dig @host để test DNS port 53)
```


Phần 3

Server Tạm Thời

Dựng listener — Không cần cài app

Tình huống: Cần verify firewall rule cho một service chưa deploy. Không cần cài nginx, python, hay bất cứ thứ gì.

```
# Terminal 1 – Bên Server (listen port 9999)
nc -l 9999          # hoặc: nc -l -p 9999 (cả hai đều dùng được)

# Terminal 2 – Bên Client
nc server_ip 9999

# Bây giờ: gõ gì ở client → hiện ở server
#           gõ gì ở server → hiện ở client
# Ctrl+C để đóng
```

Ứng dụng thực tế:

- Verify firewall rule / Security Group cho port X trước khi deploy app
- Test NetworkPolicy trong K8s
- Confirm end-to-end connectivity giữa 2 host

Server persistent — Chấp nhận nhiều kết nối

```
# netcat-openbsd (Ubuntu 24.04 default): chỉ accept 1 kết nối rồi thoát
nc -l 9999          # ← thoát sau khi client disconnect

# ncat: keep-open mode (cài trước: apt install ncat)
ncat -l 9999 --keep-open  # ← nhận kết nối tiếp theo

# Workaround với netcat-openbsd (dùng loop):
while true; do nc -l 9999; done
```

💡 Khi cần persistent listener trong production debug, dùng `ncat --keep-open` thay vì loop shell — ổn định hơn và không có race condition giữa các kết nối.

Phần 4

Transfer File

Transfer file — Không cần SSH / SCP

Tình huống: Cần copy file sang server không có SSH key setup, hoặc trong môi trường restricted (container, VM không có scp).

```
# Bước 1: Bên NHẬN chạy trước (listen)
nc -l 9999 > received_file.tar.gz

# Bước 2: Bên GỬI
nc receiver_ip 9999 < file_to_send.tar.gz

# Transfer thư mục (dùng pipe)
# Bên nhận:
nc -l 9999 | tar xzf -

# Bên gửi:
tar czf - /path/to/directory | nc receiver_ip 9999
```

⚠ Không có encryption, không có progress bar, không có checksum. Dùng để debug và di chuyển file nội bộ — không dùng qua internet.

Phần 5

HTTP Thủ Công & Banner Grabbing

HTTP thủ công — Hiểu raw HTTP

```
printf "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n" | nc example.com 80
```

Output thực tế:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Date: Thu, 24 Apr 2026 10:00:00 GMT
Server: ECS (dcb/7F5B)
...
<!doctype html>
<html>...
```

Tại sao dùng thay vì curl?

- Thấy đúng raw bytes server trả về, không có curl parsing
- Test HTTP/1.0 vs HTTP/1.1 behavior
- Debug server trả về header kỳ lạ



Cho HTTP/HTTPS debug hằng ngày → `curl -v` tiện hơn. `nc` dùng khi cần thấy tầng thô hơn curl.

Banner Grabbing — Xác định service

Kết nối TCP đến port 22/25/3306 — nhiều service tự động gửi banner khi accept kết nối.

```
# SSH — xem version
nc -v 192.168.1.1 22
# SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.6

# SMTP — xem mail server
nc -v 192.168.1.1 25
# 220 mail.example.com ESMTP Postfix (Ubuntu)

# MySQL — confirm đang chạy (không cần credentials)
nc -v 192.168.1.1 3306
# J (binary MySQL handshake packet)

# Redis
nc -v 192.168.1.1 6379
# (kết nối, gõ PING)
# +PONG
```

Dùng để: Confirm service đúng loại, đúng version trước khi debug sâu hơn.

Phần 6

Relay / Pipe

Relay — Forward traffic qua nc

Tình huống: Host A không kết nối trực tiếp được Host C, nhưng Host B kết nối được cả hai.

```
Host A  →  Host B (relay)  →  Host C:80
           port 8888
```

```
# Trên Host B: forward port 8888 → Host C port 80
mkfifo /tmp/relay-pipe
nc -l 8888 < /tmp/relay-pipe | nc host-c 80 > /tmp/relay-pipe

# Host A kết nối Host B:8888 → traffic tới Host C:80
nc host-b 8888
```

Dùng để: Debug routing, test firewall rule trong mạng phức tạp, verify packet flow giữa nhiều segment.

⚠ *Relay nc không persistent — chỉ dùng để test một lần. Cần proxy thật → dùng `socat` hoặc `nginx stream`.*

Phần 7

Kịch bản thực chiến

Scenario A: "Không biết firewall có chặn port không?"

Vấn đề phổ biến nhất: app fail connect, không rõ lỗi ở firewall hay ở app.

```
# Bước 1: Trên server target – mở listener tạm
nc -l 12345

# Bước 2: Từ client – test kết nối
nc -zv server_ip 12345

# Phân tích kết quả:
# ✓ succeeded → Firewall OK, vấn đề ở application layer
# ✗ refused → Host tới được, KHÔNG có gì listen (app chưa start hoặc sai port)
# ✗ timeout → Firewall/Security Group đang DROP packet
```

Tại sao không dùng **ping** ?

Ping test ICMP (L3). **nc -zv** test TCP port (L4) — chính xác hơn vì test đúng protocol app đang dùng. Firewall thường chặn ICMP nhưng mở TCP.

Scenario B: Debug NetworkPolicy trong Kubernetes

```
# Mục tiêu: Pod A → Pod B port 8080
# Không biết NetworkPolicy có chặn không?

# Step 1: Trong Pod B – dựng listener
kubectl exec -it pod-b -- nc -l 8080

# Step 2: Trong Pod A – test kết nối
kubectl exec -it pod-a -- nc -zv <pod-b-ip> 8080

# Kết quả:
# succeeded → NetworkPolicy OK, lỗi ở app trong Pod B
# failed → NetworkPolicy đang chặn → check egress rules của Pod A
# → check ingress rules của Pod B
```

💡 Kết hợp với `kubectl get networkpolicy -A` và kiểm tra `podSelector` / `namespaceSelector` nếu nc fail.

Scenario C: Verify port trống trước khi deploy

```
# Kiểm tra port 3000 có đang bị chiếm không  
nc -zv localhost 3000
```

```
# Connection refused → port trống ✅ an toàn start app  
# succeeded → port đang bị chiếm ❌ → dùng ss để tìm process:  
ss -tlnp | grep :3000
```

Scenario D: Test UDP (DNS, QUIC, game server)

```
# Server – listen UDP
nc -ul 5353

# Client – gửi data
echo "test" | nc -u server_ip 5353
```

Lưu ý quan trọng về UDP:

TCP: 3-way handshake → nc -z biết chắc kết nối thành công/thất bại
UDP: Không có handshake → gửi đi không biết nhận được không

- nc -zu "succeeded" chỉ nghĩa là: packet đã được gửi đi
- KHÔNG xác nhận server có nhận không, port có mở không
- Test UDP thật sự cần app-level response:
 - DNS: dig @server_ip domain (xem response)
 - NTP: ntpdate -q server_ip (xem sync)

Phần 8

Giới hạn & Khi nào dùng công cụ khác

Giới hạn của nc

Tình huống	Vấn đề với nc	Dùng thay thế
Test HTTPS / TLS	nc không handle TLS handshake	<code>openssl s_client -connect host:443</code>
Debug HTTP headers	Raw output khó đọc	<code>curl -v</code>
Persistent server	nc-traditional thoát sau 1 kết nối	<code>ncat --keep-open</code> hoặc <code>socat</code>
Monitor traffic liên tục	nc không có stats	<code>tcpdump</code>
Port scan đầy đủ	Chậm, không có service detection	<code>nmap</code>
Proxy/forward ổn định	mkfifo relay không reliable	<code>socat</code> hoặc <code>nginx stream</code>

nc vs các tool liên quan

<code>nc -zv host port</code>	→ Test port có mở không (nhANH NHẤT)
<code>ss -tlnp grep port</code>	→ Xem process nào đang listen port đó
<code>curl -v http://host:port</code>	→ Test HTTP end-to-end với headers
<code>openssl s_client</code>	→ Test TLS + xem certificate
<code>tcpdump port X</code>	→ Xem packet thực tế đi qua port
<code>nmap -sV host</code>	→ Scan + detect service version

Quy tắc chọn tool:

- Hỏi "port mở không?" → `nc -zv`
- Hỏi "ai đang dùng port?" → `ss`
- Hỏi "HTTP trả về gì?" → `curl -v`
- Hỏi "packet đi đâu?" → `tcpdump`

Key Takeaways

2 lệnh giải quyết 80% bài toán "tại sao không kết nối được":

```
# 1. Test kết nối từ client
nc -zv host port

# 2. Dựng listener tạm trên server (không cần install gì)
nc -l port
```

Đọc kết quả nc -zv:

Kết quả	Nghĩa	Hành động tiếp theo
succeeded	Port mở, firewall OK	Debug app layer
Connection refused	Port đóng, không có listener	Kiểm tra app có start không
Timeout	Firewall DROP hoặc host không tồn tại	Kiểm tra Security Group / iptables

Cảm ơn đã theo dõi! 🙏

Network Thực Chiến

Tập tiếp theo: **dig** — DNS Debug chuyên nghiệp

"`nc -zv host port` — lệnh đầu tiên khi không kết nối được. Succeeded = lỗi app. Timeout = lỗi firewall."